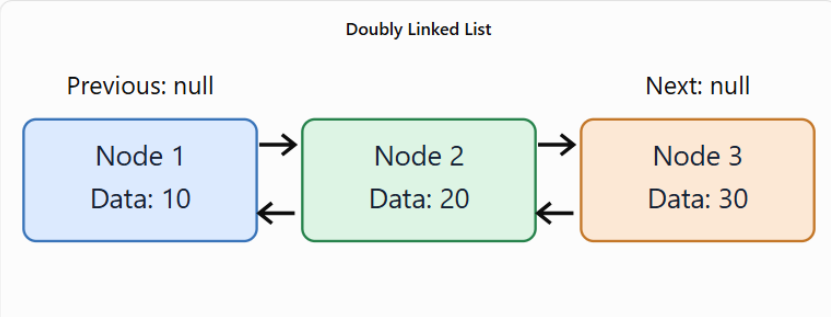
LinkedList Class in Java Collection Framework

The `LinkedList` class in Java is part of the Java Collection Framework and implements the `List` and `Deque` interfaces. It stores elements in a sequence using a doubly linked list.

Unlike an `ArrayList`, which uses a resizable array internally, a `LinkedList` uses nodes. Each node stores the element, a reference to the previous node, and a reference to the next node.

1. Internal structure of LinkedList



Each node links to both its previous and next node. The first node has no previous node, and the last node has no next node.

2. Creating a LinkedList

```
import java.util.LinkedList;

public class LinkedListDemo {
    public static void main(String[] args) {

        LinkedList<Integer> numbers = new LinkedList<>();

        numbers.add(10);
        numbers.add(20);
        numbers.add(30);

        System.out.println(numbers);
    }
}
```

Output:

```
[10, 20, 30]
```

3. Important methods of LinkedList

Method	Description
`add(element)`	Adds an element at the end
`add(index, element)`	Inserts an element at a specified index
`addFirst(element)`	Adds an element at the beginning
`addLast(element)`	Adds an element at the end
`get(index)`	Returns the element at an index
`getFirst()`	Returns the first element
`getLast()`	Returns the last element
`set(index, element)`	Replaces an element
`remove()`	Removes and returns the first element
`remove(index)`	Removes an element by index
`removeFirst()`	Removes the first element
`removeLast()`	Removes the last element
`contains(element)`	Checks whether an element exists
`size()`	Returns the number of elements
`isEmpty()`	Checks whether the list is empty
`clear()`	Removes all elements
`peek()`	Retrieves the first element without removing it
`poll()`	Retrieves and removes the first element

4. Demonstration of LinkedList methods

```
import java.util.LinkedList;

public class LinkedListMethods {
    public static void main(String[] args) {

        LinkedList<String> fruits = new LinkedList<>();

        // Adding elements
        fruits.add("Apple");
        fruits.add("Banana");
        fruits.add("Mango");
        System.out.println("Original List: " + fruits);

        // Adding at specific positions
        fruits.addFirst("Orange");
        fruits.addLast("Grapes");
        fruits.add(2, "Pineapple");
        System.out.println("After Adding: " + fruits);
```

```
        // Accessing elements
        System.out.println("First: " + fruits.getFirst());
        System.out.println("Last: " + fruits.getLast());
        System.out.println("Element    at    index    2:    "    +
fruits.get(2));

        // Updating an element
        fruits.set(1, "Papaya");
        System.out.println("After Updating: " + fruits);

        // Removing elements
        fruits.removeFirst();
        fruits.removeLast();
        fruits.remove("Mango");
        System.out.println("After Removing: " + fruits);

        // Searching and size
        System.out.println("Contains Apple: " +
                           fruits.contains("Apple"));
        System.out.println("Size: " + fruits.size());

        // Queue operations
        fruits.offer("Watermelon");
        System.out.println("Peek: " + fruits.peek());
        System.out.println("Poll: " + fruits.poll());
        System.out.println("Final List: " + fruits);
    }
}
```

5. LinkedList as a Queue

Since `LinkedList` implements the `Queue` interface, it can be used to implement a first-in, first-out (FIFO) system.

FIFO: First In, First Out

Customer A
Customer B
Customer C

Front — served first

Rear — served last

```
import java.util.LinkedList;
import java.util.Queue;
```

```
public class QueueDemo {
    public static void main(String[] args) {

        Queue<String> customers = new LinkedList<>();

        customers.offer("Amit");
        customers.offer("Neha");
        customers.offer("Rahul");

        System.out.println(customers);
        System.out.println("Next       customer:       "       +
customers.peek());

        System.out.println("Serving: " + customers.poll());
        System.out.println("Remaining: " + customers);
    }
}
```

Output:

```
[Amit, Neha, Rahul]
Next customer: Amit
Serving: Amit
Remaining: [Neha, Rahul]
```

6. LinkedList vs ArrayList

Feature	**ArrayList**	**LinkedList**
Internal structure	Resizable array	Doubly linked list
Access by index	\(O(1)\)	\(O(n)\)
Add/remove at beginning	\(O(n)\)	\(O(1)\)
Add at end	Amortized \(O(1)\)	\(O(1)\)
Insert/remove in middle	Usually \(O(n)\)	\(O(n)\) to locate; \(O(1)\) to unlink once located
Memory usage	Generally lower	Higher due to node references
Implements `List`	Yes	Yes
Implements `Deque`	No	Yes

Important: Although insertion or deletion in a `LinkedList` can be efficient once the node is located, finding an arbitrary position takes linear time. Therefore, `ArrayList` is generally a better default for list operations, while `LinkedList` can be useful for deque-style operations.

7. Practice problems

1. Create a `LinkedList<Integer>` and insert 10 numbers. Display the list in forward and reverse order.
2. Write a program to remove all occurrences of a given element from a `LinkedList`.
3. Implement a customer queue using `LinkedList` with options to add, serve, and display customers.
4. Reverse a `LinkedList` without using `Collections.reverse()`.
5. Merge two `LinkedList<Integer>` objects and sort the resulting list.
6. Remove duplicate elements from a `LinkedList` while preserving their original order.